The Journal of Systems and Software 231 (2026) 112615

Contents lists available at ScienceDirect

The Journal of Systems & Software

journal homepage: www.elsevier.com/locate/jss

# From attack descriptions to vulnerabilities: A sentence transformer-based approach☆

Refat Othman [a,*], Diaeddin Rimawi [a], Bruno Rossi [b], Barbara Russo [a]

[a] Free University of Bozen-Bolzano, Bolzano, 39100, Italy
[b] Masaryk University, Brno, 60200, Czech Republic

ARTICLE INFO

Dataset link: https://figshare.com/articles/dataset/VULDATDataSet_xlsx/25828102/1?file=46347484

*Keywords:*
Cyber threat intelligence
MITRE ATT&CK
CAPEC
CVE
Sentence transformer
Attack–vulnerability linking
Pretrained language models

ABSTRACT

In the domain of security, vulnerabilities frequently remain undetected even after their exploitation. In this work, vulnerabilities refer to publicly disclosed flaws documented in Common Vulnerabilities and Exposures (CVE) reports. Establishing a connection between attacks and vulnerabilities is essential for enabling timely incident response, as it provides defenders with immediate, actionable insights. However, manually mapping attacks to CVEs is infeasible, thereby motivating the need for automation. This paper evaluates 14 state-of-the-art (SOTA) sentence transformers for automatically identifying vulnerabilities from textual descriptions of attacks. Our results demonstrate that the `multi-qa-mpnet-base-dot-v1` (`MMPNet`) model achieves superior classification performance when using attack Technique descriptions, with an $F_1$-score of 89.0, precision of 84.0, and recall of 94.7. Furthermore, it was observed that, on average, 56% of the vulnerabilities identified by the `MMPNet` model are also represented within the CVE repository in conjunction with an attack, while 61% of the vulnerabilities detected by the model correspond to those cataloged in the CVE repository. A manual inspection of the results revealed the existence of 275 predicted links that were not documented in the MITRE repositories. Consequently, the automation of linking attack techniques to vulnerabilities not only enhances the detection and response capabilities related to software security incidents but also diminishes the duration during which vulnerabilities remain exploitable, thereby contributing to the development of more secure systems.

## 1. Introduction

Cybersecurity represents a fundamental challenge in the protection of modern systems, which are continually exposed to evolving and complex cyber threats (Admass et al., 2024). Once a cyberattack manifests, immediate action is essential to detect and mitigate the vulnerabilities exploited during an incident. When these vulnerabilities are not quickly identified and linked to the attack, systems remain exposed to malicious activities, resulting in substantial financial and operational losses. In fact, cyberattacks are expected to cost organizations $3 trillion in 2015, $6 trillion in 2021, and more than $10.5 trillion annually by 2025 (Robert Muggah, 2023). Moreover, an attacker targets an organization's system over a thousand times a week on average (Check Point, 2022). However, cybersecurity professionals can leverage Cyber Threat Intelligence (CTI) to proactively defend against cyberattacks (Rahman et al., 2023). In this context, well-structured cybersecurity knowledge repositories are crucial for effective defense. These resources assist security analysts in understanding and tracing connections between a system's vulnerabilities and the methods used to exploit them.

The MITRE Corporation has created a collection of resources, such as Adversarial Tactics, Techniques, and Common Knowledge (ATT&CK) (MITRE, 2025a), Common Attack Pattern Enumeration and Classification (CAPEC) (MITRE, 2025b), Common Weakness Enumeration (CWE) (MITRE, 2025c), and Common Vulnerabilities and Exposures (CVE) (MITRE, 2025d). ATT&CK provides tactics, techniques, and procedures (TTPs) used by cyber attackers. CAPEC represents a publicly available catalog of common attack patterns helping to understand how attackers can exploit weaknesses (MITRE, 2025b). In addition, CAPEC is structured as a list of information about the attack, including techniques for the attack, potential outcomes, and mitigations (Refat et al., 2024). CWE is a community-developed collection of typical weaknesses in software, coding errors, and security flaws. CVE is a standardized dictionary of common terms for publicly known cybersecurity vulnerabilities. All these initiatives aim to enhance the efficiency of identifying, finding, and fixing vulnerabilities by providing a unified

☆ This article is part of a Special issue entitled: 'Euromicro SEAA 2024' published in The Journal of Systems & Software.

* Corresponding author.
*E-mail address:* ramneh@unibz.it (R. Othman).

https://doi.org/10.1016/j.jss.2025.112615
Received 2 March 2025; Received in revised form 24 August 2025; Accepted 1 September 2025
Available online 9 September 2025
0164-1212/© 2025 The Authors. Published by Elsevier Inc. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

naming system (Armerding, 2017; Othman, 2024). The CVE entries are widely used, but CVE issues often lack contextual information about the attack techniques that may exploit them (Sönmez, 2021; Elder et al., 2022).

MITRE repositories such as ATT&CK, CAPEC, CWE, and CVE provide valuable and structured knowledge. However, because these repositories are normally maintained manually and evolve independently, they often lack cross-references between related entities. For example, many CVE reports contain detailed vulnerability descriptions but rarely indicate which attack techniques might exploit them, while ATT&CK entries describe adversary behaviors without explicitly citing the CVEs they could trigger. Consequently, analysts who need to trace from an observed technique to actionable CVE identifiers encounter a fragmented search process that is time-consuming and prone to omissions.

Given the scale of these resources, with 625 ATT&CK Techniques and more than 295,000 CVEs (MITRE, 2025d), manual mapping is infeasible. This burden is compounded by the complexity and rapid evolution of cyber threats, which introduce latency and inconsistencies in manual curation and leave essential links missing or outdated. As a result, vulnerabilities may remain undetected even after exploitation, and delays in linking attacks to the exploited vulnerabilities hinder timely response and remediation. Systematically linking ATT&CK techniques to CVE vulnerabilities would provide immediate, actionable insights for decision-makers. Therefore, automating this process is crucial, as it accelerates threat mitigation, supports timely decision-making, and strengthens defensive mechanisms.

A key insight underlying our work is that when an attack description and a vulnerability report share semantically similar characteristics, such as affected software components, exploited behaviors, or observable consequences, then it is possible to infer a context link between them using sentence transformers. For instance, ATT&CK Technique T1566.001 (Spearphishing Attachment) involves adversaries delivering emails with malicious file attachments to deceive users and execute payloads. This behavior semantically aligns with several CVEs that exploit similar deception techniques, including spoofed file types, misleading download dialogs, and inconsistently rendered file names (e.g., `CVE-2001-0398`, `CVE-2002-0722`, `CVE-2004-1104`, `CVE-2005-1575`, and `CVE-2005-0243`). These semantic overlaps suggest that the link between attack techniques and vulnerabilities can be inferred by analyzing the linguistic similarity between their textual descriptions. Thus, our solution leverages transformer-based sentence models, which are pre-trained to understand semantic relationships between textual inputs. These models encode both attack and vulnerability descriptions into fixed-size embeddings in a shared vector space. Computing cosine similarity between these embeddings to predict the links between attack and vulnerability descriptions can help define how closely these embeddings relate in the vector space, allowing for the discovery of meaningful links that can be missed manually. To address this challenge, our previous work introduced VULDAT (Othman et al., 2024b), a tool for linking MITRE ATT&CK techniques to CVE vulnerabilities using MPNet-based sentence embeddings. While effective, VULDAT focused only on Technique descriptions and evaluated nine transformer models.

In this paper, we propose a more comprehensive solution that extends VULDAT by evaluating 14 state-of-the-art (SOTA) transformer models for automated vulnerability detection from cyberattack text. Our approach leverages the SOTA sentence transformers to link the textual description of an attack (Tactic, Technique, Procedure, and Attack Pattern) to the textual description of CVE reports, demonstrating the model's effectiveness in both validating known links and discovering new links in the MITRE repositories. Through automation, our approach reduces the time and effort required for vulnerability identification and provides timely, actionable data that enables cybersecurity professionals to respond to threats more quickly and effectively. To validate our approach, we have built an annotated dataset with explicit links found in MITRE repositories. Our approach's dataset and source code are available on GitHub (Othman, 2025a). Thus, we intend to answer the following research questions:

- **RQ$_1$**: *(Transformers' comparison) Which combination of attack information and sentence transformer model achieves the best performance in detecting vulnerabilities?*
  To answer this question, we evaluate 14 SOTA sentence transformer models, comparing their performance in vulnerability prediction. These models, listed in Table 2, include architectures like Bidirectional Encoder Representations from Transformers (BERT) (Reimers and Gurevych, 2019) and Text-to-Text Transfer Transformer (T5) (Raffel et al., 2020), optimized for tasks such as semantic search and text classification. The goal is to find the most effective sentence transformer model for identifying attack-related textual descriptions of vulnerabilities. In addition to comparing model performance, we analyze how different types of attack descriptions influence the performance of the sentence transformers. Specifically, we examine four types of attack information derived from the MITRE ATT&CK and CAPEC repositories: Tactic, Technique, Procedure, and Attack Pattern. We hypothesize that Technique descriptions are the most informative and will yield superior results, while larger or semantically optimized transformer models are expected to outperform lightweight models due to their richer contextual embedding. By evaluating each model on these distinct information types, we aim to determine which model performs best overall and which kind of attack description most effectively supports vulnerability detection.
- **RQ$_2$**: *(Best models' effectiveness) To what extent do the best transformer models correctly detect vulnerabilities from attack descriptions?*
  To answer this question, we evaluated the overlap between the explicit links from the ground truth dataset and our detection list of CVE issues generated by our approach. Our goal was to assess how accurately the model can reproduce existing links and whether it can identify additional, potentially missing ones. We hypothesize that the best-performing models will achieve a substantial but incomplete overlap with the ground truth, as sentence transformers are expected to capture most of the documented CVE associations while also producing new links due to the inherent incompleteness of the MITRE repositories.
  To this aim, we measured performance using three metrics: Jaccard Similarity, Mapping Accuracy, and Detection Accuracy. The aim is to show our approach's in detecting CVE issues related to the textual description of attacks.
- **RQ$_3$**: *(Recommending CVEs) To what extent can our approach recommend missing links between attack techniques and vulnerabilities?*
  To address this question, we examined whether our approach can identify potential CVE links that are not explicitly available in the MITRE repositories. By analyzing cases where the model predicts CVEs not found in the ground truth, we assessed the relevance of these "missing" links. We hypothesize that our approach can uncover undocumented but semantically valid attacks to vulnerability links, indicating that MITRE repositories are incomplete and can be enriched through automated linking. Our goal is to evaluate the approach's ability to recommend semantically valid but undocumented connections that could enrich existing vulnerability databases and support proactive threat analysis.

Overall, the major contributions of our work are the following:

- A proof-of-concept approach that supports CTI research by automatically linking vulnerability descriptions from attack reports;
- An automated approach for linking attacks to vulnerabilities, validated through an evaluation of 14 SOTA sentence transformers;
- A novel annotated mapping dataset (Othman, 2025b) explicitly linking ATT&CK with vulnerabilities found in MITRE repositories;

- Empirical analysis of the impact of attack description types in vulnerability detection performance with 275 new validated links between known attacks and vulnerabilities;

The paper is structured as follows: Section 2 briefly summarizes the background, key concepts, and prior studies on vulnerabilities. Section 3 discusses the methodology, including our approach and dataset. Section 4 shows the results. Section 5 identifies the limitations and summarizes the threats to validity. Section 6 discusses the related work, and the paper concludes in Section 7.

## 2. Background

In this section, we review the key concepts relevant to our study: vulnerabilities, weaknesses, as well as attacks and the knowledge bases that have been used in this study. We also describe the sentence transformer models as they have been used extensively in the current work.

### 2.1. Vulnerability knowledge bases

There are several definitions for vulnerabilities. A **vulnerability** is defined as a flaw in software, firmware, hardware, or a service component resulting from a weakness that can be exploited, causing a negative impact on the confidentiality, integrity, or availability of an impacted component or components (Dong et al., 2023; Elder et al., 2024). NIST defines a **software vulnerability** as "a security flaw, glitch, or weakness found in software code that could be exploited by an attacker (threat source)" (Dempsey et al., 2017). A **weakness** is a condition in the software, firmware, hardware, or service components that, under certain circumstances, could contribute to introducing vulnerabilities (MITRE, 2025c; Esposito and Falessi, 2024). Weaknesses are related to vulnerabilities — in the sense that they refer to problems that can reduce the security of a system, even if no actual exploit has been identified. When an attacker finds a way to exploit a weakness, then it becomes a vulnerability. For instance, through a weakness, an attacker might be able to assume the identity of a superuser or system administrator and get complete unauthorized access and make the system vulnerable (Alevizopoulou et al., 2021; Gasmi et al., 2019; Othman and Russo, 2023). An **attack** is a way to exploit a vulnerability of a system, while several **Tactics** can be combined to represent an attack pattern for a system (Theisen et al., 2018). Since it is impossible to predict whether a weakness exists when a vulnerability will be exploited or what impact an attack will have on a system, it is of foremost importance to provide stakeholders with instruments to detect and, possibly, remove vulnerabilities (Iorga et al., 2021; Queiroz et al., 2019; Baccar, 2021; Dionísio et al., 2019; Tang et al., 2023).

When a **vulnerability** is revealed and given a CVE identifier, the vendor organization in charge of the software starts working on a patch to fix the issue. The purpose of patching, along with other strategies for addressing coding flaws, is to identify and resolve problems before attackers can exploit them (Dempsey et al., 2017). The CVE repository contains vulnerability reports from different systems and applications that can be publicly accessed (Othman, 2024; Armerding, 2017). For every publicly known vulnerability, a CVE report includes various pieces of information, including a description, an identification number (CVE-ID), and at least one reference to a system in which the vulnerability has been exploited (Sun et al., 2023), together with known vulnerability fixes and mitigation, impact ratings, and severity scores based on the Common Vulnerability Scoring System (CVSS) (NVD NIST, 2025). Table 5 shows an example of an attack and description for CWE, and CVE records. We can see the common vulnerability and exposure CVE-2022-4826 affecting the Simple Tooltips WordPress plugin prior to version 2.1.4. This vulnerability is related to improper validation and escaping of shortcode attributes before rendering them on a page or post. As a result, users with contributor roles and above could inject malicious scripts, leading to Stored Cross-Site Scripting (XSS) attacks. This vulnerability can be linked to the Pattern of **attack** CAPEC-38 with an attacker attempting to load a malicious resource into the program so that it will be executed.

CWE is a community-developed collection of typical weaknesses in software, coding errors, and security flaws. A CWE report includes various information such as a name, an identification number (CWE-ID), a description, an observed example containing CVE reports ID related to CWE, and related attack patterns highlighting the relationships between specific attack patterns and CWE. CWE and CVE reports are connected through the CVE-ID.

### 2.2. Attack knowledge bases

We leveraged the information on attacks contained in the ATT&CK and CAPEC repositories. ATT&CK (MITRE, 2025a) is a framework that provides a comprehensive inventory of **tactics**, **techniques**, and **procedures** adopted by attackers during various stages of a cyber-attack (Rahman et al., 2024; Son et al., 2023; Irshad and Siddiqui, 2023). TTPs help security experts understand adversary behavior, guide threat detection and response efforts, and enhance organizational defenses against cyberthreats. According to the ATT&CK model, **tactics** represent the goal of an adversary's attack, like ways to get access to a secured network, whereas **techniques** define how to carry out an attack in the context of a certain Tactic, like using phishing attempts (Mitre, 2025; Satvat et al., 2021). Techniques can be further detailed into **subtechniques**. **Procedures** provide detailed insights into how adversaries effectively use the techniques in the actual attacks. For example, a procedure could involve sending emails camouflaged as legitimate communications from a trusted source to trick users into clicking malicious links. Researchers typically use TTPs to profile or examine the attack life cycle on a specific system. Our study used Version 14 of the ATT&CK framework, which contains 14 Tactics, 201 Techniques, 424 Subtechniques, and 809 procedures. The framework also provides the ATT&CK Matrix, which breaks each attack down into tactics, techniques, and procedures. An example of the relations among Tactics, Techniques, Subtechniques, and Procedures are shown in Fig. 1. All relations are many-to-many. For instance, the Technique *"event triggered execution"* has 16 Subtechniques, whereas, *"account access removal, acquire access, and audio capture"* has no Subtechnique. The Common Attack Pattern Enumeration and Classification (CAPEC) (MITRE, 2025b) is a catalog of common **Attack Patterns**. CAPEC is linked to CWE using the CWE-ID, creating an association between specific attack techniques and the underlying weaknesses they exploit (MITRE, 2025a). Attack Patterns allow us to see what techniques in the ATT&CK repository an adversary could use to target the weaknesses and vulnerabilities. For instance, the Pattern*"privilege escalation (CAPEC-233)"* elevates attackers' privilege to perform an action they are not supposed to be authorized to perform. Related to this Pattern, one can link the Technique *"Abuse Elevation Control Mechanism (T1548)"* that describes the techniques that attackers use to gain higher-level permissions on a system or network. Among those, there is the Technique *"Access token manipulation (T1134)"*, which can be implemented in Metasploit (Rapid7, 2025; MITRE, 2025) with the procedure *"named-pipe impersonation"* (Mitre, 2025; Wu et al., 2021; Noor et al., 2019). We will connect ATT&CK with CAPEC in our dataset, matching the techniques' IDs.

The attack descriptions in these repositories vary significantly in their structure, length, and level of detail. For example, ATT&CK Tactics are abstract and short, while CAPEC Patterns are typically longer and more technically detailed. Table 1 summarizes the typical characteristics of the four types of attack descriptions. Manually linking 625 ATT&CK techniques to over 295,000 CVEs is impractical. To address this, we employ sentence transformers to enable a fully automated and semantically driven integration across ATT&CK, CAPEC, CWE, and CVE repositories.

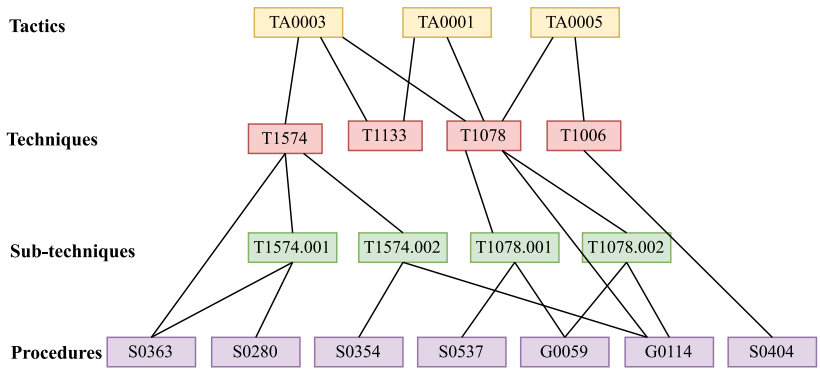


**Fig. 1.** An example of the ATT&CK relations.

**Table 1**
Typical characteristics of the attack types.

| | Tactic | Technique | Procedure | Pattern |
|---|---|---|---|---|
| Definition | High-level goal of an adversary | General method of how the adversary achieves a goal | Specific instance of an attacker using a technique | Structured description of a recurrent method of attack |
| Vocabulary | Abstract, it uses mostly generic terms | It is more specific, but it still uses reusable terms | It includes tool names, scripts, commands | It uses technical terms, exploit-specific language |

**Table 2**
Overview of SOTA Sentence Transformer Models per architecture and ordered by embedding dimensions.

| Acronym | Model | Architecture | Embedding Dimension | Model size |
|---|---|---|---|---|
| PAlbert | paraphrase-albert-small-v2[1] | ALBERT | 768 | 43 MB |
| PTinyBERT | paraphrase-TinyBERT-L6-v2[2] | TinyBERT | 768 | 240 MB |
| MDBERT | multi-qa-distilbert-cos-v1[3] | DistilBERT | 768 | 250 MB |
| MSMBERT | msmarco-bert-base-dot-v5[4] | BERT | 768 | 420 MB |
| DRoBERTa | all-distilroberta-v1[5] | Distil-RoBERTa | 768 | 290 MB |
| Roberta | all-roberta-large-v1[6] | RoBERTa | 1024 | 1.36 GB |
| MiniLM6 | all-MiniLM-L6-v2[7] | MiniLM | 384 | 80 MB |
| MiniLM12 | all-MiniLM-L12-v2[8] | MiniLM | 384 | 120 MB |
| MMiniLM6 | multi-qa-MiniLM-L6-cos-v1[9] | MiniLM | 384 | 80 MB |
| PMiniLM6 | paraphrase-MiniLM-L6-v2[10] | MiniLM | 384 | 80 MB |
| PMiniLM12 | paraphrase-multilingual-MiniLM-L12-v2[11] | MiniLM | 384 | 420 MB |
| MPNet | all-mpnet-base-v2[12] | MPNet | 768 | 420 MB |
| MMPNet | multi-qa-mpnet-base-dot-v1[13] | MPNet | 768 | 420 MB |
| XXLT5 | gtr-t5-xxl[14] | T5-XXL | 4096 | 9.23 GB |

[1] PAlbert:https://huggingface.co/sentence-transformers/paraphrase-albert-small-v2
[2] PTinyBERT:https://huggingface.co/sentence-transformers/paraphrase-TinyBERT-L6-v2
[3] MDBERT:https://huggingface.co/sentence-transformers/multi-qa-distilbert-cos-v1
[4] MSMBERT:https://huggingface.co/sentence-transformers/msmarco-bert-base-dot-v5
[5] DRoBERTa:https://huggingface.co/sentence-transformers/all-distilroberta-v1
[6] Roberta:https://huggingface.co/sentence-transformers/all-roberta-large-v1
[7] MiniLM6:https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2
[8] MiniLM12:https://huggingface.co/sentence-transformers/all-MiniLM-L12-v2
[9] MMiniLM6:https://huggingface.co/sentence-transformers/multi-qa-MiniLM-L6-cos-v1
[10] PMiniLM6:https://huggingface.co/sentence-transformers/paraphrase-MiniLM-L6-v2
[11] PMiniLM12:https://huggingface.co/sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2
[12] MPNet:https://huggingface.co/sentence-transformers/all-mpnet-base-v2
[13] MMPNet:https://huggingface.co/sentence-transformers/multi-qa-mpnet-base-dot-v1
[14] XXLT5:https://huggingface.co/sentence-transformers/gtr-t5-xxl

### 2.3. Sentence transformers

Sentence transformer models are pre-trained models that produce sentence embeddings for various natural language processing tasks, such as semantic search, paraphrasing, and clustering. However, these models often contain millions of parameters, making fine-tuning and deployment challenging due to latency and capacity constraints. In this work, we utilize 14 SOTA pre-trained sentence transformer models, as summarized in Table 2. These models are recognized as top performers in semantic similarity (Muennighoff et al., 2022; Colangelo et al., 2025; Choi et al., 2021; UKPLab, 2024). All models are sourced from the Hugging Face Model Hub,[1] which provides a comprehensive repository of transformer-based models for natural language processing tasks. The selected models have been extensively benchmarked and fine-tuned for sentence-level tasks, such as semantic textual similarity, information retrieval, and clustering (Muennighoff et al., 2022; Colangelo et al., 2025; Choi et al., 2021; UKPLab, 2024). Notably, models like MPNet, MiniLM, MSMARCO, RoBERTa, and T5 consistently achieve strong results across benchmarks (Muennighoff et al., 2022; Colangelo et al., 2025). Lightweight models, including PAlbert, have also demonstrated competitive performance in the evaluations (Choi et al., 2021). A summary table listing these models along with their architecture, embedding size, and benchmark performance for semantic similarity is available in the official sentence transformers documentation (UKPLab, 2024), supporting the transparency and reproducibility of our model selection. In addition, we selected models with different architectures and examined how these architectural differences affect performance in linking attack descriptions to CVEs.

BERT (Reimers and Gurevych, 2019) is a pre-trained transformer model designed for natural language understanding. It is based on a deep bidirectional architecture, capturing context from both left and right of a word, which significantly enhances performance on tasks like text classification, question answering, and more. The BERT-based models include `MSMBERT`, optimized for semantic search, as well as `PAlbert` and `PTinyBERT`, which are smaller versions that utilize parameter reduction techniques while still maintaining competitive performance.

DistilBERT and DistilRoBERTa, on which `MDBERT` and `DRoBERTa` are based, compress BERT and RoBERTa, respectively, using knowledge distillation, reducing model size by 40% while retaining 97% of their accuracy, making them suitable for real-time applications. RoBERTa enhances BERT by removing next-sentence prediction and training on more data, leading to better generalization. Additionally, MiniLM architecture (Wang et al., 2020) models such as `MMiniLM6`, `MiniLM6`, `PMiniLM12`, `MiniLM12`, and `PMiniLM6`, further reduce computational complexity by employing a minimalistic self-attention mechanism. These models are a new version of the BERT model (Reimers and

[1] HuggingFace:https://huggingface.co/

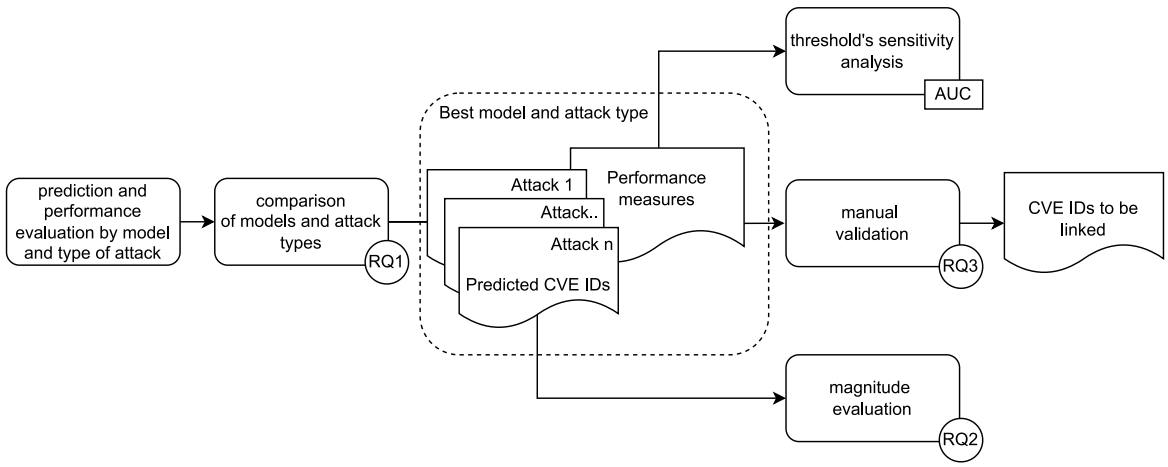


**Fig. 2.** Overview of the methodology for linking attack descriptions to CVE reports.

Gurevych, 2019), a popular model for natural language understanding, that has been created using knowledge distillation (Hinton et al., 2015) to compress the large model BERT (called the teacher model) into a small model MiniLM (called the student model), which uses much fewer parameters and computations while achieving competitive results on downstream tasks. MiniLM notably reduces the effort for finding the most similar pairs while maintaining the accuracy of BERT. Moreover, they employ a 6 and a 12-layer neural network to generate vectors that encapsulate the text. Unlike BERT, which learns from a single language, these models are trained on a diverse dataset from 50 different languages. The MPNet-based model, `MMPNet` and `MPNet`, extend BERT by integrating XLNet's permutation-based training, improving contextual understanding and retrieval accuracy. T5 (Raffel et al., 2020), exemplified by `XXLT5` model, is designed with a transformer architecture and operates on large-scale text-to-text tasks, allowing it to be used for summarization, translation, classification, and more, making it highly versatile across applications. In this study, we use pre-trained transformer models to generate embeddings for attack and vulnerability descriptions, and then apply a similarity layer based on cosine similarity to determine whether an attack is linked to a vulnerability (Section 3.1.4).

## 3. Methodology

Fig. 2 overviews our approach. Firstly, for each model (sentence transformer) and type of attack text (Tactic, Technique, Procedure, or Attack Pattern), we perform a *vulnerability prediction and model's performance analysis* as described in Section 3.1, to generate ranked lists of potential CVE matches and quantify model effectiveness. Secondly, to answer $RQ_1$, we compare models and attack types by their performance on the corresponding attacks as illustrated in Section 3.2, identifying the model–attack-type combination that achieves the highest prediction accuracy. Then, with the best model and attack type, we perform (1) a *sensitivity analysis* of the threshold used to determine the predicted vulnerabilities as illustrated in Section 3.3, to identify the threshold value that yields the best trade-off between precision and recall, (2) an evaluation of the number of correctly predicted vulnerabilities per attack text (*magnitude evaluation*) to answer $RQ_2$ in Section 3.4, and (3) a *manual validation* of the predicted vulnerabilities to discuss $RQ_3$, Section 3.5, to demonstrate their practical value for expanding MITRE's ATT&CK–CVE mappings and reducing manual effort.

### 3.1. Vulnerability prediction and model's performance analysis

Fig. 3 shows how we use a sentence transformer to predict vulnerabilities from an attack text of a given type. We collect all attack descriptions related to that type, along with all vulnerability descriptions and their IDs (*texts and links collection*). For each attack ID, we associate the CVE IDs eventually found in the attack's description page (links) (*attacks annotation*). Then, we pre-process all texts (*text pre-processing*). After fine-tuning the model, we embed all texts with the selected model (*text embedding*). The embedding of each attack text is compared for similarity with the embeddings of all vulnerabilities (*text similarity analysis*). Vulnerability embeddings that have a similarity score greater than the threshold with a given attack embedding are labeled as predicted vulnerabilities for that attack. In this way, we obtain a mapping between the attack IDs and CVE IDs. At this point, we evaluate the performance of the sentence transformer by comparing the sets of predicted and linked CVE IDs. In the next step, we compare the performance by varying the model and attack type. In the following, we describe this process in more detail.

#### 3.1.1. Texts and links collection

This step systematically collects and structures attack descriptions and vulnerability records from MITRE repositories to provide a transparent, reproducible foundation and ground truth for evaluating the automated linking approach. From the MITRE repositories, we first collect the descriptions of the attacks of all types as well as the descriptions of all vulnerabilities. By inspecting the repositories' web pages, we also retrieve any links that explicitly connect attacks and vulnerabilities (*explicit link*). For example, on the web page of the weakness CWE-770 there exist 10 different explicit links to CVEs and we map CWE-770 to the corresponding CVE IDs. The graph in Fig. 4 illustrates the result of such analysis. Each node clusters attacks, weaknesses and vulnerabilities, and its value indicates the number of explicit links found in the respective web pages of a node of another type. Some nodes in the graph remain isolated, referred to as "floating entries", and the majority of CVE reports are represented within this category. The portion of floating entries somehow measures the lack of knowledge about the relation between attacks and vulnerabilities. On the other hand, there are "Super Entries", nodes with many outgoing or incoming edges. The portion of these nodes represents the more mature knowledge of attacks, vulnerabilities, and their relations. In addition, the directions of the graph edges indicate where the links have been found. For instance, there are 447 CWE web pages that include links to 1405 CVEs. As Procedures are only listed in the Techniques' pages, there are also no explicit links between Patterns or Tactics and Procedures. Therefore, we omit the Procedures' nodes. Table 3 reports the number of CVEs and CWEs that are linked or not to attacks. In particular, the table shows that a large majority of vulnerabilities are not explicitly linked in any attack page. This may be due to the fact that we leverage the explicit links between CWE and CVE to link attacks to

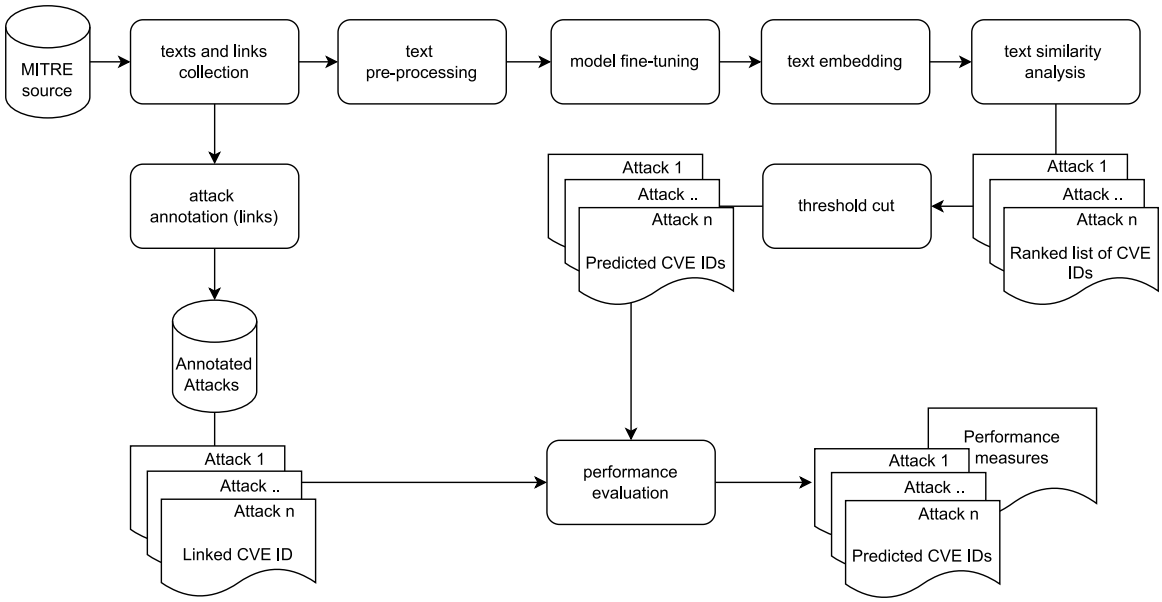


**Fig. 3.** Workflow for prediction and performance evaluation by model and attack type.

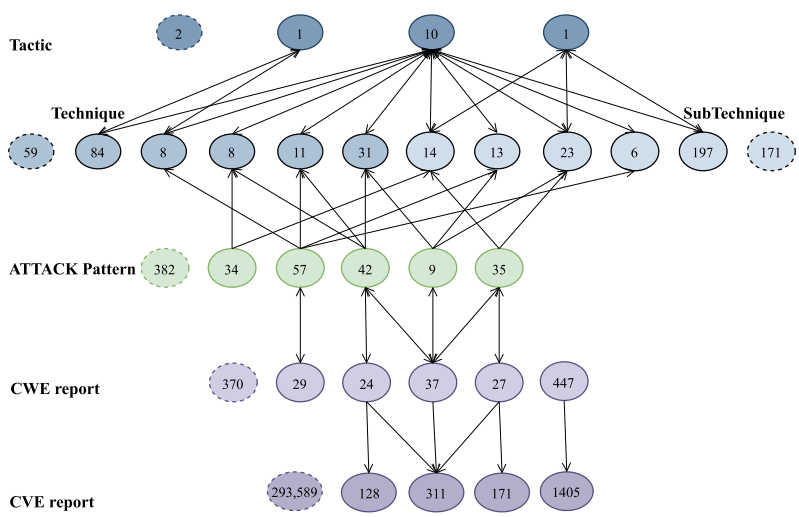


**Fig. 4.** Graph representation of explicit links between attack types, weaknesses, and vulnerabilities in MITRE repositories, showing the density and direction of existing connections.

**Table 3**
Number of CVEs and CWEs linked and not linked to attacks.

| | | Tactic | Technique | Procedure | Attack pattern |
|---|---|---|---|---|---|
| CWE | linked | 117 | 79 | 117 | 117 |
| | not linked | 818 | 856 | 818 | 818 |
| CVEs | linked | 610 | 439 | 610 | 610 |
| | not linked | 294 994 | 295 165 | 294 994 | 294 994 |

**Table 4**
Attacks linked and not linked to CVE reports.

| | Tactic | Technique | Procedure | Attack pattern |
|---|---|---|---|---|
| Linked | 11 | 100 | 721 | 86 |
| Not linked | 3 | 525 | 88 | 473 |
| Total | 14 | 625 | 809 | 559 |

vulnerabilities. As noted in the CWE pages, the explicit links to CVEs is *"a curated list of examples for users to understand the variety of ways in which this weakness can be introduced. It is not a complete list of all CVEs that are related to this CWE entry"*. Thus, some existing links may not be reported in the page.

### 3.1.2. Attacks' annotation

We utilize explicit links to establish connections between attacks and a defined set of vulnerabilities. As illustrated in Fig. 4, there are no explicit links between Tactics, Techniques, or Procedures and CWEs or CVEs. We then exploit the explicit links between CWEs and attack patterns to annotate attacks with the CVEs to which they are linked. This step creates a traceable mapping between attacks and vulnerabilities, forming the ground truth for model evaluation. For instance, given a Technique, we retain in CAPEC all attack patterns that mention it. For each Attack Pattern, we then collect all the CWEs whose explicit link mention it, and from those CWEs, we retrieve the explicit links to CVEs. Following this procedure, Table 4 illustrates the attacks that are linked to some CVEs.

**Table 5**
An example of a technique and its chain to one of the linked CVEs.

| | | | |
|---|---|---|---|
| Technique-T1574.007: Adversaries may execute their own malicious payloads by hijacking environment variables used to load libraries. Adversaries may place a program in an earlier entry in the list of directories stored in the PATH environment variable, which Windows will then execute when it searches sequentially through that PATH listing in search of the binary that was called from a script or the command line. | CAPEC-38: This Pattern of attack sees an adversary load a malicious resource into a program's standard path so that when a known command is executed then the system instead executes the malicious component. The adversary can either modify the search path a program uses, like a PATH variable or classpath, or they can manipulate resources on the path to point to their malicious components. If one of these libraries and/or references is controllable by the attacker then application controls can be circumvented by the attacker. | CWE-427: The product searches for critical resources using an externally-supplied search path that can point to resources that are not under the product's direct control. | CVE-2022-4826: The Simple Tooltips WordPress plugin before 2.1.4 does not validate and escape some of its shortcode attributes before outputting them back in a page/post where the shortcode is embed, which could allow users with the contributor role and above to perform Stored Cross-Site Scripting attacks. |

Table 5 illustrates an example of such links in the case of the Technique T1574.007, Attack Pattern CAPEC-38, CWE-427, and CVE-2022-4826. The table includes only one representative instance among all those that can be linked to Technique T1574.007 using our procedure.

Finally, we annotate each attack ID by the CVE IDs we link with our procedure. For instance, in the above example, Technique ID T1574.007 is annotated with CVE-2022-4826. In the general case, an attack ID can be annotated with several CVEs IDs. The annotated dataset can be found in our replication package (Othman, 2025b) and is used as ground truth for the performance analysis in Section 3.1.6. Using this annotation, for each attack ID $a$, we build the *set of linked CVE IDs*:

$$\mathcal{M}(a) = \{c \in \mathcal{C} \ : \ \exists\, a \rightarrow c\}$$

where $\mathcal{C}$ is the set of all CVE IDs and $a \rightarrow c$ indicates a link between the attack ID $a$ and the CVE ID $c$ found with our procedure.

### 3.1.3. Text pre-processing

We pre-processed the attack and CVE descriptions to remove irrelevant information and textual noise while retaining semantic content. The text was normalized to lowercase, and citations, URLs, and general non-alphanumeric characters were removed. Unlike traditional NLP pipelines, we did not apply stemming, lemmatization, or stop-word removal, as prior studies (Okonkwo et al., 2023; Siino et al., 2024) have shown that transformer-based models rely on subword tokenization and contextual embeddings, which benefit from preserving grammatical structure and functional words. Stop words, in particular, contribute to the syntactic flow of sentences and often enhance the contextual understanding that attention-based models exploit. Similarly, avoiding stemming and lemmatization ensures that morphological variants are preserved, allowing the model to capture subtle differences in meaning. This minimal preprocessing balances noise reduction with semantic preservation, enabling the sentence transformer to better capture relationships between attack descriptions and CVE reports.

### 3.1.4. Model fine-tuning and text embedding

We have implemented our approach using 14 pre-trained sentence transformer models (as discussed in Section 2.3), with embedding sizes ranging from 384 to 4096 dimensions. These models are designed to produce sentence embeddings, fixed-size vector representations of input texts. Specifically, both attack and vulnerability descriptions are transformed into vectors in a shared vector space, allowing for their comparison. Each of the selected models uses a multi-layer architecture to process input text. The tokens of each text are passed through multiple transformer encoder layers, which employ self-attention mechanisms and feed-forward networks to capture the contextual relationships among the tokens. Afterward, a pooling strategy such as mean pooling or using the [CLS] token is applied to condense the token-level embeddings into a fixed-length sentence embedding. For model fine-tuning, we applied a single split to our dataset into three subsets with ratios 80%, 10%, and 10% and apply the model for training, validation, and testing, respectively, as recommended in Hiniduma et al. (2025). This fixed split was chosen to ensure fair comparability between models, avoid variability from multiple random splits, and maintain reproducibility. Dataset statistics before splitting, including linked and unlinked samples for each attack type, are reported in Table 4. All models were fine-tuned using CosineSimilarityLoss. Training was conducted for four epochs with 100 warmup steps, and evaluation was performed every 500 steps, following established practices in sentence transformer training examples (UKPLab, 2025). The validation set was used for intermediate evaluation, and the test set for final performance reporting.

### 3.1.5. Similarity analysis

Following established practices in embedding-based NLP systems (Reimers and Gurevych, 2019; Muennighoff et al., 2022), we adopt cosine similarity as the primary metric for comparing sentence embeddings in transformer-based models. This choice aligns with its well-established role in embedding-based machine learning systems, where it has consistently demonstrated effectiveness across a range of natural language processing tasks. For instance, Reimers and Gurevych (Reimers and Gurevych, 2019) showed that applying cosine similarity in Sentence-BERT significantly improved performance in semantic textual similarity benchmarks, while Muennighoff et al. (2022) adopted cosine similarity as the standard metric across more than 50 models, validating its robustness for tasks such as information retrieval, clustering, and semantic search. In this work, we perform cosine similarity (Eq. (1)) on the normalized output vectors of an attack embedding $p = (p1, p2, \ldots, pn)$ with all CVE embeddings $q = (q1, q2, \ldots, qn)$. Cosine similarity is a common metric in natural language processing and information retrieval because it captures the angular relationship between vectors, focusing on their direction rather than magnitude. This property makes it especially effective for comparing sentence embeddings, where semantic meaning is primarily represented by the vector's orientation in the embedding space. Cosine similarity scores theoretically range from −1 to 1, where 1 indicates maximum similarity, 0 indicates no correlation, and −1 indicates complete opposition. Since we normalize embeddings and focus exclusively on semantically related pairs, our practical range of interest is [0, 1]. For consistency with the threshold selection procedure, these values are expressed on a 0–100 scale in our experiments. This property makes cosine similarity especially effective for comparing sentence embeddings, where semantic meaning is primarily represented by the vector's orientation in the embedding space.

$$Sim(\vec{p} \cdot \vec{q}) = \frac{\vec{p} \cdot \vec{q}}{\|\vec{p}\| \cdot \|\vec{q}\|} = \frac{\sum_{i=1}^{n} p_i q_i}{\sqrt{\sum_{i=1}^{n} p_i^2} \cdot \sqrt{\sum_{i=1}^{n} q_i^2}} \tag{1}$$

We rank all CVEs' embeddings by the similarity score with the attack embedding. A threshold $\rho$ is applied to cut the resulting ranked list at a certain level of similarity. Thus, we associate to an attack ID $a$ a list of CVE IDs whose similarity score is greater than the threshold:

$$\mathcal{L}_{\rho}(a) = \{c \in C \ : \ Sim(\vec{a} \cdot \vec{c}) > \rho\}$$

**Table 6**
Classification in the attacks' set $\mathcal{A}$.

| Type | Description |
|---|---|
| Positives | $\{a \in \mathcal{A} : \exists c \in \mathcal{M}(a)\}$ |
| Negatives | $\{a \in \mathcal{A} : \nexists c \in \mathcal{M}(a)\}$ |
| Predicted Positives | $\{a \in \mathcal{A} : \exists c \in \mathcal{L}_\rho(a)\}$ |
| Predicted Negatives | $\{a \in \mathcal{A} : \nexists c \in \mathcal{L}_\rho(a)\}$ |
| True Positives (TP) | $\{a \in \mathcal{A} : \mathcal{L}_\rho(a) \cap \mathcal{M}(a) \neq \emptyset\}$ |
| False Positives (FP) | $\{a \in \mathcal{A} : (\mathcal{L}_\rho(a) \neq \emptyset \wedge \mathcal{M}(a) = \emptyset)\}$ |
| False Negatives (FN) | $\{a \in \mathcal{A} : (\mathcal{L}_\rho(a) = \emptyset \wedge \mathcal{M}(a) \neq \emptyset)\}$ |
| True Negatives (TN) | $\{a \in \mathcal{A} : (\mathcal{L}_\rho(a) = \emptyset \wedge \mathcal{M}(a) = \emptyset)\}$ |

where $C$ is the set of all CVE IDs. $\mathcal{L}_\rho(a)$ is the set of *predicted CVEs IDs* for an attack ID $a$ and a threshold $\rho$. With $\mathcal{L}_\rho$, we can define a classification problem: a model predicts a vulnerability from an attack if the set $\mathcal{L}_\rho(a)$ for the attack ID $a$ is non-empty and it does not if the set is empty.

#### 3.1.6. Performance evaluation

We can evaluate the performance of a model on the above classification problem. A perfect model should satisfy the following condition for any attack ID $a$: either the intersection between the predicted and the linked CVE IDs is non-empty,

$$\mathcal{L}_\rho(a) \cap \mathcal{M}(a) \neq \emptyset,$$

or both sets are empty:

$$\mathcal{L}_\rho(a) = \emptyset \wedge \mathcal{M}(a) = \emptyset.$$

Table 6 defines the classification problem in the attack set.

We compute the performance metrics by means of the cardinality of the sets in Table 6. Being our approach a classifier, we use Precision, Recall, and their harmonic mean $F_1$ to evaluate its performance:

$$Precision = \frac{TP}{TP + FP} \tag{2}$$

$$Recall = \frac{TP}{TP + FN} \tag{3}$$

$$F_1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{4}$$

The $F_1$-score indicates how well the categorization task performs in terms of both precision and recall: the higher the $F_1$-score, the better. We fine-tune each of the pre-trained models on our dataset. To this aim, the same 80/10/10 train, validation and test split described in Section 3.1.4 was applied to all models to ensure consistency and a fair comparison. For each attack type, we evaluate the performance of each model in the testing set using Precision, Recall, and $F_1$-Score. The experiments are conducted on a six-node GPU cluster, where each node is equipped with an NVIDIA A100 (80 GB), 192 GB of RAM, and an Intel Xeon 4208 (16-core) CPU. The code was implemented using PyTorch 2.8.0 and Hugging Face Transformers 4.55.2.

### 3.2. Comparison of models and attack types

To answer $RQ_1$, we compute Precision, Recall and $F_1$ as described in Section 3.1.6, for each of the 14 models in Table 2 and all attack types (Tactic, Technique, Procedure and Pattern). In addition, this evaluation is to test our hypothesis that Technique descriptions are more informative than other attack types for vulnerability detection. Moreover, we hypothesize that transformer models with larger capacity and richer semantic representations will outperform lighter architectures. Finally, we use the maximum value of $F_1$ to determine the best model(s) and the attack type(s) on which the model(s) predict best. The output is pairs (attack type, model) where the model best performs on the set of that type of attack.

### 3.3. Threshold's sensitivity analysis

To identify the predicted CVE IDs (Section 3.1), we initially set the similarity threshold $\rho = 60\%$ based on our initial manual understanding of the samples. To validate our choice, we therefore perform a *sensitivity analysis* by varying the threshold $\rho$ and re-evaluating the model's performance as recommended in Lobo et al. (2022), Sheng and Ling (2006). We select the best (attack type, model) pair identified in the comparison analysis in Section 3.2. Then, we randomly sample a balanced subset from the original dataset of that attack type, consisting of 50 positive and 50 negative examples, on which we study the classification problem. We employ the Receiver Operating Characteristic (ROC) curve to plot the True Positive Rate (TPR)

$$TPR = \frac{TP}{TP + FN} \tag{5}$$

vs. the False Positive Rate (FPR)

$$FPR = \frac{FP}{TN + FP} \tag{6}$$

across a range of threshold values from 1 to 100. The threshold for which the ROC value achieves the closest value (Euclidean distance) to the values (0,1) – perfect classification – will be selected. The Area Under the ROC Curve (AUC) serves as a comprehensive measure of model performance across all thresholds, where a value of 1.0 indicates perfect classification, and 0.5 corresponds to performance equivalent to random guessing. To ensure fair and consistent comparison, the same threshold value $\rho$ was uniformly applied across all evaluated models.

### 3.4. Magnitude evaluation

To answer $RQ_2$ and evaluate the overlap between the predicted and actual links in our dataset, we adopt three set-based metrics that provide complementary insights into prediction quality. These metrics help us quantify how many relevant links were correctly identified by our model and how well the predicted set matches the actual set. In particular, we hypothesize that the best-performing models will reproduce a substantial portion of the ground truth CVE links, although the overlap will remain incomplete due to the inherent limitations of the repositories. To test this, we employ three metrics: Jaccard Similarity Index, Mapping Accuracy, and Detection Accuracy. The Jaccard Similarity evaluates the similarity of two sets: the detected set ($\mathcal{L}_a$) and the actual sets ($\mathcal{M}_a$). It measures the ratio of the intersection of these sets to their union,

$$\text{Jaccard Similarity} = \frac{|\mathcal{L}_a \cap \mathcal{M}_a|}{|\mathcal{L}_a \cup \mathcal{M}_a|} \tag{7}$$

Mapping Accuracy measures the portion of predicted CVE IDs in the set of linked CVE IDs and is the ratio of the intersection of the predicted set ($\mathcal{L}_a$) and the actual set ($\mathcal{M}_a$) to the size of the actual set ($\mathcal{M}_a$).

$$\text{Mapping Accuracy} = \frac{|\mathcal{L}_a \cap \mathcal{M}_a|}{|\mathcal{M}_a|} \tag{8}$$

Detection Accuracy measures the portion of linked CVE IDs in the set of predicted CVE IDs and is the ratio of the intersection of the predicted set ($\mathcal{L}_a$) and the actual set ($\mathcal{M}_a$) to the size of the predicted set ($\mathcal{L}_a$).

$$\text{Detection Accuracy} = \frac{|\mathcal{L}_a \cap \mathcal{M}_a|}{|\mathcal{L}_a|} \tag{9}$$

These metrics provide a comprehensive understanding of the model's ability to accurately detect and map links between the predicted and actual links in the ground truth.

### 3.5. Manual validation

To answer $RQ_3$, we select the best pair(s) (attack type, model) resulting from the comparison in Section 3.2. We select the same balanced sample we use for the sensitivity analysis in Section 3.3 In

**Table 7**
Performance metrics for all types of attacks and models.

| Acronym | Tactic | | | Technique | | | Procedure | | | Pattern | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Precision | Recall | $F_1$-Score | Precision | Recall | $F_1$-Score | Precision | Recall | $F_1$-Score | Precision | Recall | $F_1$-Score |
| PAlbert | 50 | 66.7 | 57.1 | 96.8 | 22.6 | 36.6 | 100 | 6.9 | 12.8 | 47.8 | 50 | 48.9 |
| PTinyBERT | 100 | 18.2 | 30.8 | 100 | 22.6 | 36.8 | 92.1 | 19.1 | 31.6 | 53.3 | 46.5 | 49.7 |
| MDBERT | 75 | 27.3 | 40 | 100 | 15 | 26.1 | 70.8 | 15.2 | 24.1 | 37.5 | 3.5 | 6.4 |
| MSMBERT | 50 | 100 | 66.7 | 66.5 | 100 | 79.9 | 50 | 100 | 66.7 | 48.6 | 100 | 65.4 |
| DRoBERTa | 66.7 | 18.2 | 28.6 | 79.7 | 41.4 | 54.5 | 77.8 | 8 | 14.4 | 52.1 | 43 | 47.1 |
| Roberta | 75 | 27.3 | 40 | 84.8 | 66.9 | 74.8 | 83.3 | 17 | 28.3 | 52.3 | 53.5 | 52.9 |
| MiniLM6 | 100 | 18.2 | 30.8 | 86.7 | 29.3 | 43.8 | 100 | 1.1 | 2.2 | 48.6 | 20.9 | 29.3 |
| MiniLM12 | 100 | 33.3 | 50 | 94.5 | 51.9 | 66.9 | 100 | 3.4 | 6.5 | 52.8 | 32.6 | 40.3 |
| MMiniLM6 | 100 | 9.1 | 16.7 | 95.2 | 15 | 26 | 4.4 | 60 | 3.5 | 50 | 10.5 | 17.3 |
| PMiniLM6 | 40 | 66.7 | 50 | 93.3 | 31.6 | 47.2 | 86.9 | 2.8 | 5.4 | 52 | 75.6 | 61.6 |
| PMiniLM12 | 40 | 66.7 | 50 | 93.7 | 44.4 | 60.2 | 76.9 | 22.7 | 35 | 57.2 | 87.3 | 69.1 |
| MPNet | 100 | 9.1 | 16.7 | 77.1 | 83.5 | 80.1 | 97.5 | 44.3 | 60.9 | 52.9 | 53.5 | 53.2 |
| MMPNet | 100 | 33.3 | 50 | 84 | 94.7 | **89.0** | 89.7 | 29.5 | 44.4 | 71.6 | 73.3 | **72.4** |
| XXLT5 | 78.6 | 100 | **88.0** | 66.5 | 100 | 79.9 | 48.6 | 100 | 65.4 | 50.1 | 100 | 66.7 |

line with our hypothesis that the model can uncover undocumented but semantically valid attacks to vulnerability links, we analyze the descriptions of the CVEs predicted by the model for these attacks. In particular, we focus on all CVE IDs that were predicted but not explicitly linked to the attacks (False Positives). This set includes both the wrongly predicted CVE IDs (i.e., the "truly" false positives) and the CVE IDs that should have been explicitly linked on the MITRE pages but are instead missing (the imperfect oracle built through the MITRE pages, Section 3.1.2). The first and last authors performed a manual iterative validation on the missing CVE IDs. Missing CVE IDs were considered "to be linked" when both authors agreed that there was a clear semantic match between the Technique and the CVE descriptions after two iterations.

## 4. Results and discussion

In this section, we present the results of our experiments by the outcomes for each of the RQs, as follows.

### 4.1. $RQ_1$: *(Transformers' comparison) Which combination of attack information and sentence transformer model achieves the best performance in detecting vulnerabilities?*

To answer $RQ_1$, we evaluate the performance of the 14 sentence transformer models in Table 7 over different types of attacks as described in Section 3.1.6. Specifically, we compute their performance on the description of (1) Tactic, (2) Technique, (3) Procedure, and (4) Pattern. Table 7 presents the comparative analysis across all models and types of attacks. In terms of models, MMPNet achieves the highest $F_1$-score for both Technique (89.0) and Attack Pattern (72.4). T5 also performs well, achieving the best $F_1$-score in Tactic (88.0) and consistently good results across other attack types with 100% recall scores, though at the expense of lower precision in some cases (e.g., Procedure: Precision= 48.6). For what concerns the attack types, Techniques are the most informative attack type for most models, with many achieving $F_1$-scores above 60%. MMPNet is the best model for this type. Procedure is the less informative attack type for the sentence transformers with MSMBERT the best performing model.

*Sensitivity analysis.* As the results in Table 7 are computed using a heuristic threshold of $\rho = 60\%$, we further perform a sensitivity analysis (Section 3.3) on the threshold using the best pair (Technique, MMPNet). We perform the analysis on a balanced dataset of Techniques. The dataset comprises 50 positive and 50 negative instances randomly selected from the original dataset. Fig. 5 shows the ROC curve and the optimal value of the threshold $\rho$. The value of AUC = 0.82 indicates a good overall discriminative ability. In this paper, we used the same similarity threshold $\rho$ value for all models during evaluation. The selection of the threshold embodies the trade-off between

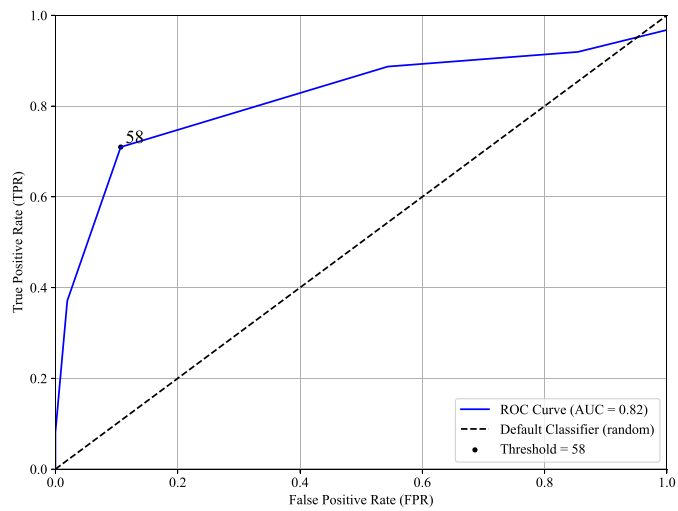


**Fig. 5.** ROC of varying threshold $\rho$ for pair (MMPNet, Technique). The value nearest to the ideal ratios (0,1) corresponds to $\rho = 58\%$.

precision and recall: higher recall reduces the likelihood of missing true vulnerabilities, while higher precision decreases false alarms and analyst workload. Although the ROC analysis $\rho = 58\%$ confirmed the consistency of our initial choice, we acknowledge that employing a single global threshold across all models and attack types constitutes a limitation of this study in Section 5. In practice, individual models or attack types may benefit from tailored thresholds that better capture their specific operating characteristics and optimize the precision–recall balance.

*Discussion.* XXLT5 is a large-scale variant of Google T5 designed for dense retrieval tasks, such as semantic search (Ni et al., 2021). In our study, it appears to be the most appropriate to detect similarity between Tactics – that have a very abstract description – and CVEs. This model embeds texts in the largest vector space and has the highest number of parameters among all 14 models in Table 2. Such characteristics allow the model to capture more complex patterns and semantic similarities, although the model is more computationally expensive than the others. MMPNet is a much thinner and faster model based on the MPNet architecture. Nonetheless, it embeds text in a comparatively large vector space and has a rather large size among the models in Table 2. The superior performance of MPNet can be explained by its architecture, which leverages permuted language modeling to better capture bidirectional dependencies and contextual semantics compared to other models. In our study, this is the best model for both Techniques and Patterns. This is coherent with the fact that both Techniques and

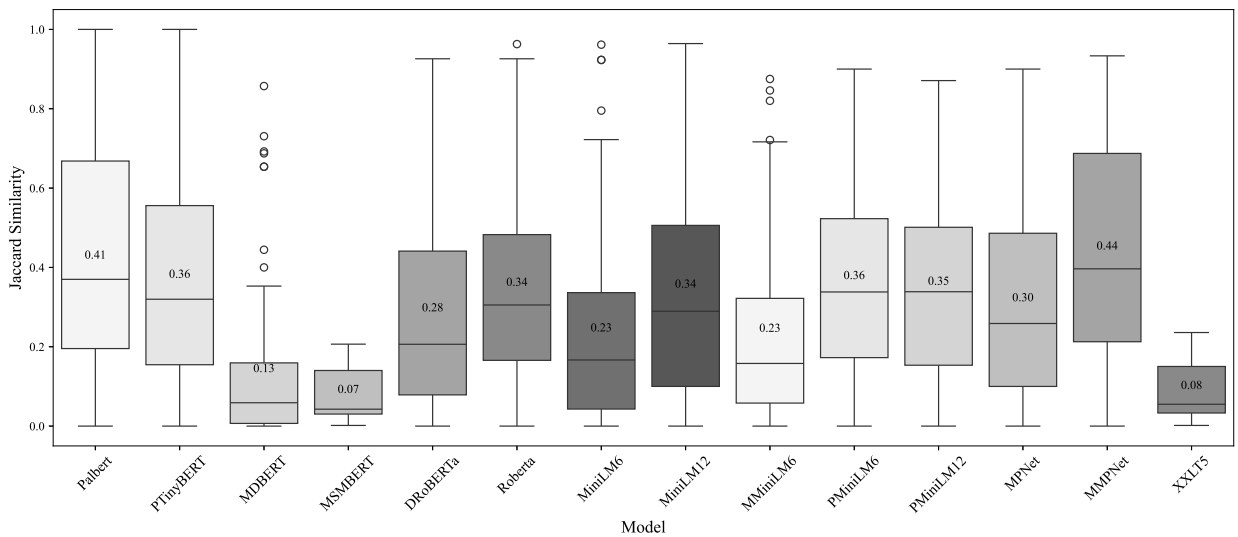


**Fig. 6.** Jaccard similarity across SOTA models.

Patterns refer to exploitation methods, Table 1. Interestingly, only MMPNet, together with MSMBERT uses the dot product (instead of cosine) in their loss functions. This allows us to account for the length of the embeddings, not only their angle, and thus they can capture the differences in embeddings based on their context semantics. It is also worth noting that the other dot product model, MSMBERT, is the best model for Procedures. Thus, these results confirm our initial hypothesis for this research question, as Technique descriptions indeed emerged as the most informative attack type, and higher-capacity transformer models demonstrated superior performance compared to lighter architectures.

> **$RQ_1$ Summary:** Overall, sentence transformers are effective in identifying CVEs from attack descriptions, particularly for Techniques, where the MMPNet model achieves an $F_1$-score of 89. Our study further indicates that the best-performing models either leverage a high-dimensional embedding space with a large number of parameters (e.g., XXLT5), or utilize dot product loss functions, enabling more precise capture of the semantic context between attack and CVE descriptions.

### 4.2. *$RQ_2$: (Best models' effectiveness) To what extent do the best transformer models correctly detect vulnerabilities from attack descriptions?*

To answer $RQ_2$, we evaluate, for each Technique ID, how many of the predicted CVE IDs are actually linked. This evaluation began with a comparative analysis of SOTA transformer-based models, using Jaccard Similarity as the primary metric (Fig. 6). The figure indicates that MMPNet achieved the highest Jaccard Similarity (mean = 0.44), demonstrating superior capability in identifying linked CVE IDs from attack descriptions. Based on this observation, MMPNet was selected for further assessment using three complementary metrics, Jaccard Similarity, Mapping Accuracy, and Detection Accuracy, as defined in Section 3.4.

Fig. 7 reports the distribution of these measures across all Techniques. On average, the Mapping and Detection accuracies report more than random guessing (50%), but also report a large amount of undetected linked CVE IDs (44% on average) and a good amount of predicted unlinked CVE IDs (39%). For more than 50% of the Techniques the model predicts CVE IDs that are over 70% linked (i.e., a median Detection Accuracy above 70%). Conversely, for more than 50% of Techniques, over 50% of their linked CVE IDs are correctly predicted. A representative example is Technique T1539 "Steal Web Session Cookie" that achieves the highest Jaccard Similarity score of 82% obtained as the ratio between the 124 CVE IDs in $\mathcal{L}(T1539) \cap \mathcal{M}(T1539)$ and the 151 in $\mathcal{L}(T1539) \cup \mathcal{M}(T1539)$. As $\mathcal{L}$ contains 150 CVE IDs and $\mathcal{M}(T1539)$ contains 125 CVE IDs, the capability of MMPNet is very high (82% and 99%). The model finds all but one existing linked CVE IDs and a good number of additional ones. The additional predicted CVE IDs may be false positives as well as potential missing CVE IDs, i.e. links that may not yet be documented. The next research question aims to address the issue of the imperfect oracle and the potential missing links. Thus, these findings confirm our hypothesis for this research question, since the best-performing models reproduced a significant portion of existing CVE links while leaving space for additional, potentially missing ones.

**Table 8**
Summary of our approach validation process for new candidates' links.

| | Retrieved | | Validated | |
|---|---|---|---|---|
| | Total predicted | False positives | False recommended | Recommended to be linked |
| Number of linked CVE reports | 2230 | 434 | 159 | 275 (12.3%) |
| Number of techniques | 201 | 74 | 54 | 60 (29.8%) |

> **$RQ_2$ Summary.** The overlapping between predicted and linked CVE IDs is satisfactory, reaching a Mapping Accuracy of 56%, Detection Accuracy of 61%, and Jaccard Similarity of 40%. Inspecting the metrics for representative cases we noticed that some predicted CVEs may be missing in the MITRE pages and can eventually be added.

### 4.3. *$RQ_3$: (Recommending CVEs) To what extent can our approach recommend missing links between attack techniques and vulnerabilities?*

Table 8 presents the CVEs predicted by our model for a selected subset of 100 technique descriptions. We summarize the unlinked Techniques predicted with the model MMPNet. For 201 Techniques

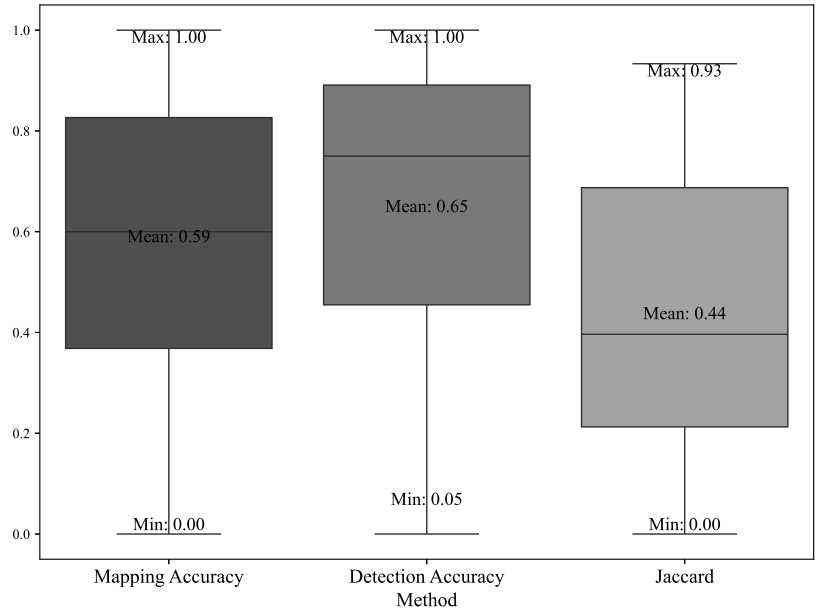


**Fig. 7.** Performance of the MMPNet-based approach across attack techniques description.

we found 434 predicted and unlinked CVE IDs. Among the latter, after the manual inspection (Section 3.5), 159 were confirmed as falsely predicted and the remaining 275 predicted by 60 Techniques as to be linked.[2] The full list of the 275 manually validated links is available as supplementary material at GitHub (Othman, 2025a). As an illustrative example, we discuss the recent story of Technique T1039 "Data from Network Shared Drive" and the CVEs our approach predicted, namely CVE-2005-1205, CVE-2009-3107, and CVE-2020-3452. All these vulnerabilities involve unauthorized access or data exposure through shared network resources, which aligns well with the description of T1039. Although these links are now included in the ATT&CK repositories, they were missing when we annotated the attack datasets, as in Section 3.1.2. In other words, we were able to uncover missing CVE IDs that were independently added to the MITRE repository later on. This further reinforces the validity of our approach. We have already established contact with members of the MITRE board and are in the process of preparing a formal submission of our manually validated links to support the enrichment of MITRE repositories. These results show that our approach successfully identified 275 previously undocumented attacks to vulnerability links across 60 Techniques, many of which were later verified or added to the MITRE repositories. This confirms our hypothesis for $RQ_3$, as it demonstrates that the current repositories are incomplete and can indeed be enriched through automated linking.

> **$RQ_3$ Summary**: Our approach is able to recommend 275 missing links across 60 Techniques, highlighting its potential to enhance the existing information in the MITRE repositories and increase coverage by over 12%.

### 4.4. Implications for practice and research

This study evaluates 14 SOTA transformers across four attack description types (i.e., *Tactic*, *Technique*, *Procedure*, and *Attack Pattern*). We show that Technique descriptions yield the highest predictive performance (MMPNet: $F_1$=89), and we report *275* previously undocumented Technique–CVE links. Taken together, these results provide empirical evidence that *manual maintenance and independently evolving repositories are insufficient to achieve timely and comprehensive cross-referencing at this scale*, thereby motivating automation. By coupling comparative model evidence with a curated set of 275 new links, this work provides both a practical pathway for decision-makers to accelerate mitigation and a research scaffold for advancing scalable, transparent, and operationally relevant attack–vulnerability linking. The following details the main implications for both practice and research fields.

*Implications for practice.* The evidence provides actionable guidance for security operations and vulnerability management: (i) prioritize Technique descriptions when predicting candidate CVEs, since they carry the strongest signal for linking; (ii) operationalize high-performing models (e.g., MMPNet) to pre-populate candidate CVE sets for each observed technique, thereby reducing analyst time on initial triage; and (iii) integrate the *275 validated new links* into vulnerability management systems and threat intelligence platforms to enrich lookups and shorten patching lead times; Collectively, these practices support earlier mitigation, more consistent triage, and stronger defensive posture.

*Implications for research.* The findings from this study and the comparative evaluation across 14 transformers and four attack information types establish a replicable baseline for automated attack-to-vulnerability linking. Building on this baseline, promising directions include: (i) model evolution, assessing newer architectures and domain-specific or multilingual models for robustness across heterogeneous cyber threat datasets. This could involve testing the approach's adaptability across broader contexts and refining its performance in varied operational environments; (ii) data breadth, extending beyond ATT&CK–CVE to additional feeds and assessing generalization under distribution shift; (iii) timeliness, moving toward near real-time ingestion of attack reports (news, advisories) to recommend relevant CVEs as events unfold, improving global threat awareness and response speed; and (iv) transparency, incorporating continuous data streams from multiple intelligence feeds, retraining models to adapt to emerging vulnerabilities, and developing interpretability and visualization tools

[2] It is worth noticing here that one CVE report can be linked to more than one Technique. Thus, the Technique numbers in the last two columns do not sum up to the value of the second column

for model outputs are also important areas for future research. Collectively, these directions provide a foundation for scalable, transparent, and operationally relevant vulnerability–attack linking systems that can both advance academic understanding and enhance real-world cybersecurity practices.

## 5. Threats to validity

In this section, we discuss the potential threats to the validity of our study in terms of construct, internal, and external validity.

**Construct validity.** Construct validity refers to how much the results at the operational level support the claims at the conceptual level (Sjøberg and Bergersen, 2022). In our case, two main construct threats arise: the choice of cosine similarity threshold ($\rho$) and the input size for transformer models. The similarity threshold $\rho$ is not absolute. We empirically determined the value that balances false positives and false negatives, but different values can have an impact on the results, knowing that a larger $\rho$ results in a more conservative approach in determining true values, but leading to a lower recall. In contrast, a smaller $\rho$ may yield a more optimistic outcome when determining true values, increasing recall, but at the expense of precision. To validate our choice, we employed ROC analysis, which confirmed the robustness of the selected threshold. Nevertheless, using a single global threshold across all models and attack types may not be optimal, as individual models could benefit from tailored thresholds that better capture their operating characteristics. Additionally, transformer models are constrained to processing a fixed number of input tokens, the first 384 words, which could potentially lead to context truncation and bias toward initial content. Nevertheless, this limitation does not affect our study, as the attack and vulnerability descriptions used are all within the 384-token limit, not impacting the connection to our conceptual representation of semantic links.

**Internal validity.** Internal validity refers to the extent to which treatment and outcome variables are related to the causal relation between the treatment and outcome constructs (Sjøberg and Bergersen, 2022; Shadish et al., 2002). In this work, the attack and vulnerability texts may contain technical terms or acronyms (e.g., software versions or specific library names), which may introduce noise in the semantic representation and influence model performance. To mitigate this, we applied consistent pre-processing steps to clean and normalize the input texts. Another potential threat arises from our use of MITRE repositories to construct the ground truth, which may evolve over time. To reduce this impact, we used the most recent version of each repository available at the time of the study and documented our data snapshot.

**External validity.** If there is a causal relationship between the construct and the effect, external validity tells us that the results can be generalized outside the scope of the study (Wohlin et al., 2012). In our case, external validity concerns mainly the generalizability of our findings to other contexts or datasets. This study faces two external threats: First, the ground truth was constructed using a subset of 100 ATT&CK subtechniques linked to 610 CVE reports. While this sample is suitable for the purposes of our exploratory analysis, it does not reflect the entire scope of the MITRE repositories, which include over 201 techniques and more than 295,000 CVEs. Second, both the ATT&CK and CVE repositories are continuously updated, meaning that the structure, content, and relationships within the data may evolve over time. As a result, models trained on a static snapshot may not maintain the same level of performance on future data. To enhance the generalizability of our approach, future work will explore integrating additional sources such as ExploitDB (OffSec, 2025) and will assess the approach on other publicly available threat intelligence datasets.

## 6. Related work

In this section, we review the related work on vulnerability attack models. Existing studies are grouped into two categories: vulnerability-to-attack mapping and attack-to-vulnerability mapping.

**Vulnerability-to-Attack Mapping:** The majority of research studies have focused on linking vulnerabilities to attacks and not the other way around. The multi-head deep embedding model (Kuppa et al., 2021) learns links between CVE reports and ATT&CK techniques. Using regular expressions in the attack reports. Then, it compares the ATT&CK Technique vectors with the text description found in the CVE metadata using the cosine distance. This study only maps 17 ATT&CK techniques. Researchers have used the BERT pre-trained transformer model (Devlin et al., 2019) to extract information from the vulnerability database to improve the textual descriptions of newly discovered CVE reports (Sun et al., 2021). A multi-label classification approach (Lakhdhar and Rekhis, 2021) introduced for automatically mapping CVE reports to ATT&CK tactics in the MITRE repositories evaluated several machine learning algorithms to find the best approach. Another model mapping between vulnerabilities and attacks is Cve2att&ck (Grigorescu et al., 2022). The model annotates a dataset of CVE reports with ATT&CK tactics, using BERT-based language models and classic models for machine learning. The study is limited to a set of 31 ATT&CK techniques and a training set of 1813 CVE reports. The CVE Transformer (CVET) (Ampel et al., 2021) is a model that combines a self-awareness distillation design utilized for fine-tuning the pre-trained language model RoBERTa (Liu et al., 2019). The main objective is linking a CVE to one of 10 ATT&CK tactics.

**Attack-to-vulnerability mapping:** The majority of research studies have focused on utilizing attack patterns in CAPEC rather than leveraging TTP information from the MITRE repositories. Based on the similarity between the CAPEC document and the CVE description, NLP-based techniques are also utilized to establish the link between CAPEC documents and CVE reports (Kanakogi et al., 2021; Hemberg et al., 2022) with RoBERTa outperforming the general BERT model. Other NLP techniques are used to build a direct correlation between CVE and CAPEC reports, such as TF–IDF (Ramos et al., 2003) and Doc2Vec (Lau and Baldwin, 2016). The optimal link is provided by the TF–IDF when using CAPEC information and CVE descriptions, also providing the *top-n* CAPEC documents that match the CVE description (Othman et al., 2024a; Kanakogi et al., 2022). To further improve attack-to-vulnerability mapping, we developed VULDAT (Othman et al., 2024b), an automated tool leveraging the MPNet sentence transformer to link attack technique descriptions to CVE vulnerabilities. Unlike previous approaches that focused solely on CAPEC patterns, VULDAT exclusively utilizes attack Technique descriptions to establish links with vulnerabilities. BRON (Hemberg et al., 2021) is a bi-directional aggregated data graph that supports relational path tracing between CWE, CVE, CAPEC, and ATT&CK tactics and techniques. Through investigations of the resulting graph representation and data mining of the relational connections between all these cybersecurity knowledge sources, BRON builds a graph framework that promises to integrate all scattered data. However, the model fails because it cannot link the most recent CVE reports to ATT&CK Enterprise Matrix techniques.

A comparison of representative prior works is summarized in Table 9. Previous studies have primarily focused on linking CVEs to a single type of attack information, such as Tactics or Techniques (Hemberg et al., 2022; Ampel et al., 2021; Grigorescu et al., 2022). For example, CVET (Ampel et al., 2021) achieved an $F_1$-score of 76.2% when mapping CVEs to ATT&CK Tactics, while Cve2att&ck (Grigorescu et al., 2022) reported an $F_1$-score of 47.8% for CVE-to-Technique mappings. Similarly, Kanakogi et al. (2021) achieved an $F_1$-score of 44.5% when linking CVEs to CAPEC attack patterns. Our earlier work, VULDAT (Othman et al., 2024b), attained an $F_1$-score of 85% for linking Technique descriptions to CVEs. In this paper, we conduct a comprehensive evaluation of 14 SOTA sentence transformer models.

**Table 9**
Summary of related work in vulnerability–attack mapping.

| Study | Approach | Key Contribution | Limitations |
|---|---|---|---|
| Kuppa, A. et al. (Hemberg et al., 2022) | Multi-head Deep: Cosine similarity on embeddings. | Links CVEs to 17 ATT&CK techniques using ATT&CK and CVE datasets. | Limited scope, regex dependency. |
| Grigorescu, Octavian, et al. (Grigorescu et al., 2022) | Cve2att&ck: BERT-based classification. | Annotates CVEs with tactics using 1,813 CVEs and 31 ATT&CK techniques. | Small training set, narrow Technique coverage. |
| Ampel, Benjamin, et al. (Ampel et al., 2021) | CVET: RoBERTa fine-tuning. | Self-awareness distillation for Tactic mapping using 10 ATT&CK tactics. | Only 10 tactics, no technique-level granularity. |
| Kanakogi, Kenta, et al. (Kanakogi et al., 2021) | CAPEC-CVE: TF–IDF, RoBERTa, Doc2Vec. | Top-$N$ CAPEC matches for CVEs using CAPEC and CVE datasets. | Unidirectional (CVE→CAPEC). |
| Othman et al. (Othman et al., 2024a) | CAPEC-CVE | Empirical comparison models (TF–IDF, LSI, BERT, MiniLM, RoBERTa) and provides a mapping dataset linking 133 CAPEC attack patterns to 685 CVEs via CWEs. | TF–IDF and LSI fail to capture contextual or semantic relationships in attack descriptions. |
| Hemberg, Erik, et al. (Hemberg et al., 2021) | BRON: graph aggregation. | Bidirectional relational path tracing using CWE, CVE, CAPEC, and ATT&CK datasets. | Fails on recent CVEs, no NLP integration. |
| Othman et al. (Othman et al., 2024b) | Technique–CVE. | Empirical comparison transformers models (BERT, MPNeT, MiniLM) and provides a mapping dataset linking 100 attack techniques to 610 CVEs via CWEs and CAPEC. | Limited in its ability to compare all types of attack information (Tactic, Technique, Procedure, and Attack Pattern). |
| Othman et al. (This study) | Att&ck2Cve: Transformer-based approach for linking attacks to CVEs across four attack information types (Tactic, Technique, Procedure, Attack Pattern) using cosine similarity on embeddings. | Evaluation of 14 SOTA sentence transformers, accompanied by a comprehensive analysis that identifies the most effective attack information type (Technique) for vulnerability prediction; the MMPNet model achieves an $F_1$-score of 89% and enables the identification of 275 validated missing CVE links, thereby enriching the MITRE repositories. | Although the limitations of related work have been addressed, we acknowledge that further improvements remain possible, as discussed in Section 5. |

Additionally, we assess the performance of these models across four distinct types of attack descriptions: Tactic, Technique, Procedure, and Attack Pattern, providing a systematic comparison of model effectiveness. Our evaluation reports that Technique descriptions are the most informative for vulnerability prediction and that MMPNet achieves an F1-score of 89%. Furthermore, our method in this article also incorporates a manual validation stage, leading to the identification of 275 undocumented Technique–CVE links, thereby enriching the MITRE repositories and enhancing their utility for cyber threat intelligence.

## 7. Conclusion and future work

In this study, we introduced a novel approach utilizing 14 sentence transformer models to automatically identify CVE reports from textual descriptions of adversary behaviors, also known as attacks. Our approach relies on the MITRE family of repositories, which represents a comprehensive knowledge base of adversary tactics and techniques as input information for CVE prediction. We constructed an annotated dataset leveraging the information contained in such repositories. We further conducted an experiment to assess the performance of each model using different types of information about an attack stored in the MITRE repositories. The results indicate that our approach performs the best when using the Technique descriptions as input, achieving an $F_1$-score of 89%. Our approach can supply a valuable service to the cybersecurity community by providing the first approach to automatically link attacks to vulnerabilities and their mitigation strategies, rather than the other way around. Our manual validation of our approach discovered 275 links between attacks and vulnerabilities not contained in the MITRE repositories. These correspond to 12.3% of the total links found by our model.

We plan to evaluate the performance of transformer models on other annotated datasets. Additionally, we will explore how these models can predict CVE issues from publicly available cyberattack news reports, such as those published by online cybersecurity magazines (e.g., SecurityWeek (SecurityWeek, 2025)). These sources typically describe real-world incidents and emerging threats in unstructured text, providing valuable context for vulnerability detection and enabling the early identification of vulnerabilities before they are exploited. Furthermore, we plan to extend our approach by conducting a study that establishes links between attacks, vulnerabilities, and weaknesses in code. Additionally, we aim to identify and provide techniques that can be used to exploit these weaknesses in code. Finally, we have already begun engaging with the MITRE board to discuss integrating our newly identified links into their existing listings.

## CRediT authorship contribution statement

**Refat Othman:** Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Formal analysis, Data curation. **Diaeddin Rimawi:** Writing – review & editing, Validation, Methodology. **Bruno Rossi:** Writing – review & editing, Validation, Supervision, Formal analysis. **Barbara Russo:** Writing – review & editing, Validation, Supervision, Formal analysis.

## Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Refat Othman reports financial support was provided by Free University of Bozen-Bolzano. Barbara Russo reports financial support was provided by Free University of Bozen-Bolzano. Diaeddin Rimawi reports financial support was provided by Free University of Bozen-Bolzano. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgments

The authors extend their gratitude to the CSLab at the Free University of Bozen-Bolzano for supporting this work under project number EFRE1039 as part of the EFRE-FESR 2021–2027 program.

## Data availability

The dataset used in this study is publicly available at Figshare: https://figshare.com/articles/dataset/VULDATDataSet_xlsx/25828102/1?file=46347484.

## References


Admass, W.S., Munaye, Y.Y., Diro, A.A., 2024. Cyber security: State of the art, challenges and future directions. Cyber Secur. Appl. 2, 100031.

Alevizopoulou, S., Koloveas, P., Tryfonopoulos, C., Raftopoulou, P., 2021. Social media monitoring for IoT cyber-threats. In: 2021 IEEE International Conference on Cyber Security and Resilience. CSR, IEEE, pp. 436–441.

Ampel, B., Samtani, S., Ullman, S., Chen, H., 2021. Linking common vulnerabilities and exposures to the mitre att&ck framework: A self-distillation approach. 1st KDD Work. AI Enabled Cybersecur. Anal..

Armerding, T., (2017). CVE Definitions.https://www.csoonline.com/article/3204884/what-is-cve-its-definition-and-purpose.html.

Baccar, K., 2021. Automated mapping of CVE vulnerabilties to MITRE ATT&CK Framework (Ph.D. thesis). Tekup.

Check Point, (2022). 38% Increase in 2022 Global Cyberattacks. https://blog.checkpoint.com/2023/01/05/38-increase-in-2022-global-cyberattacks/.

Choi, H., Kim, J., Joe, S., Gwon, Y., 2021. Evaluation of bert and albert sentence embedding performance on downstream nlp tasks. In: 2020 25th International Conference on Pattern Recognition. ICPR, IEEE, pp. 5482–5487.

Colangelo, M.T., Meleti, M., Guizzardi, S., Calciolari, E., Galli, C., 2025. A comparative analysis of sentence transformer models for automated journal recommendation using PubMed metadata. Big Data Cogn. Comput. 9 (3), 67.

Dempsey, K., Eavy, P., Moore, G., 2017. Automation support for security control assessments. Vol. 1: Overv. 1–112.

Devlin, J., Chang, M.-W., Lee, K., Toutanova, K., 2019. Bert: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers). pp. 4171–4186.

Dionísio, N., Alves, F., Ferreira, P.M., Bessani, A., 2019. Cyberthreat detection from twitter using deep neural networks. In: 2019 International Joint Conference on Neural Networks. IJCNN, IEEE, pp. 1–8.

Dong, Y., Tang, Y., Cheng, X., Yang, Y., 2023. DeKeDVer: A deep learning-based multi-type software vulnerability classification framework using vulnerability description and source code. Inf. Softw. Technol. 163, 107290.

Elder, S., Rahman, M.R., Fringer, G., Kapoor, K., Williams, L., 2024. A survey on software vulnerability exploitability assessment. ACM Comput. Surv. 56 (8), 1–41.

Elder, S., Zahan, N., Shu, R., Metro, M., Kozarev, V., Menzies, T., Williams, L., 2022. Do I really need all this work to find vulnerabilities? An empirical case study comparing vulnerability detection techniques on a java application. Empir. Softw. Eng. 27 (6), 154.

Esposito, M., Falessi, D., 2024. VALIDATE: A deep dive into vulnerability prediction datasets. Inf. Softw. Technol. 107448.

Gasmi, H., Laval, J., Bouras, A., 2019. Information extraction of cybersecurity concepts: An LSTM approach. Appl. Sci. 9 (19), 3945.

Grigorescu, O., Nica, A., Dascalu, M., Rughinis, R., 2022. Cve2att&ck: Bert-based mapping of cves to mitre att&ck techniques. Algorithms 15 (9), 314.

Hemberg, E., Kelly, J., Shlapentokh-Rothman, M., Reinstadler, B., Xu, K., Rutar, N., O'Reilly, U.-M., 2021. Linking threat tactics, techniques, and patterns with defensive weaknesses, vulnerabilities and affected platform configurations for cyber hunting. arXiv:2010.00533.

Hemberg, E., Srinivasan, A., Rutar, N., O'Reilly, U.-M., 2022. Sourcing language models and text information for inferring cyber threat, vulnerability and mitigation relationships.

Hiniduma, K., Byna, S., Bez, J.L., 2025. Data readiness for AI: A 360-degree survey. ACM Comput. Surv. (ISSN: 0360-0300) 57 (9), http://dx.doi.org/10.1145/3722214, URL https://doi.org/10.1145/3722214.

Hinton, G., Vinyals, O., Dean, J., 2015. Distilling the knowledge in a neural network. Stat 1050, 9.

Iorga, D., Corlatescu, D.-G., Grigorescu, O., Sandescu, C., Dascalu, M., Rughinis, R., 2021. Yggdrasil—early detection of cybernetic vulnerabilities from Twitter. In: 2021 23rd International Conference on Control Systems and Computer Science. CSCS, IEEE, pp. 463–468.

Irshad, E., Siddiqui, A.B., 2023. Cyber threat attribution using unstructured reports in cyber threat intelligence. Egypt. Informatics J. 24 (1), 43–59.

Kanakogi, K., Washizaki, H., Fukazawa, Y., Ogata, S., Okubo, T., Kato, T., Kanuka, H., Hazeyama, A., Yoshioka, N., 2021. Tracing cve vulnerability information to capec attack patterns using natural language processing techniques. Information 12 (8), 298.

Kanakogi, K., Washizaki, H., Fukazawa, Y., Ogata, S., Okubo, T., Kato, T., Kanuka, H., Hazeyama, A., Yoshioka, N., 2022. Comparative evaluation of NLP-based approaches for linking CAPEC attack patterns from CVE vulnerability information. Appl. Sci. 12 (7), 3400.

Kuppa, A., Aouad, L., Le-Khac, N.-A., 2021. Linking cve's to mitre att&ck techniques. In: Proceedings of the 16th International Conference on Availability, Reliability and Security. pp. 1–12.

Lakhdhar, Y., Rekhis, S., 2021. Machine learning based approach for the automated mapping of discovered vulnerabilities to adversial tactics. In: 2021 IEEE Security and Privacy Workshops. SPW, IEEE, pp. 309–317.

Lau, J.H., Baldwin, T., 2016. An empirical evaluation of doc2vec with practical insights into document embedding generation. In: Proceedings of the 1st Workshop on Representation Learning for NLP. Association for Computational Linguistics.

Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V., 2019. Roberta: A robustly optimized bert pretraining approach. arXiv preprint arXiv:1907.11692.

Lobo, A., Oliveira, P., Sampaio, P., Novais, P., 2022. Cost-sensitive learning and threshold-moving approach to improve industrial lots release process on imbalanced datasets. In: International Symposium on Distributed Computing and Artificial Intelligence. Springer, pp. 280–290.

MITRE, (2025). ATTACK. https://attack.mitre.org/.

MITRE, (2025). CAPEC. https://capec.mitre.org/.

MITRE, (2025). CWE Dataset. https://cwe.mitre.org/.

MITRE, (2025). CVE. https://www.cve.org.

Mitre, (2025). MITRE ATT&CK.https://www.mitre.org/.

MITRE, (2025). FIN6 group. https://attack.mitre.org/groups/G0037/.

Muennighoff, N., Tazi, N., Magne, L., Reimers, N., 2022. Mteb: Massive text embedding benchmark. arXiv preprint arXiv:2210.07316.

Ni, J., Qu, C., Lu, J., Dai, Z., Ábrego, G.H., Ma, J., Zhao, V.Y., Luan, Y., Hall, K.B., Chang, M.-W., Yang, Y., 2021. Large dual encoders are generalizable retrievers. arXiv:2112.07899.

Noor, U., Anwar, Z., Amjad, T., Choo, K.-K.R., 2019. A machine learning-based FinTech cyber threat attribution framework using high-level indicators of compromise. Future Gener. Comput. Syst. 96, 227–242.

NVD NIST, (2025). CVSS.https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator.

OffSec, (2025). Exploit-DB. https://www.exploit-db.com/.

Okonkwo, O., Dridi, A., Vakaj, E., 2023. Leveraging word embeddings and transformers to extract semantics from building regulations text. In: Proceedings of the 11th Linked Data in Architecture and Construction Workshop.

Othman, R.T., 2024. Vulnerability detection for software-intensive system. In: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering. pp. 510–515.

Othman, R., 2025a. Att&ck2vul - automated vulnerability detection from cyberattack text. https://github.com/ref3t/Attack2VUL/tree/main..

Othman, R., 2025b. VULDAT- vulnerability DataSet. http://dx.doi.org/10.6084/m9.figshare.25828102.v1, figshare. Dataset..

Othman, R., Rossi, B., Russo, B., 2024a. A comparison of vulnerability feature extraction methods from textual attack patterns. In: 2024 50th Euromicro Conference on Software Engineering and Advanced Applications. SEAA, IEEE, pp. 419–422.

Othman, R., Rossi, B., Russo, B., 2024b. Cybersecurity defenses: Exploration of CVE types through attack descriptions. In: 2024 50th Euromicro Conference on Software Engineering and Advanced Applications. SEAA, IEEE, pp. 415–418.

Othman, R., Russo, B., 2023. VULDAT: Automated vulnerability detection from cyber-attack text. In: Silvano, C., Pilato, C., Reichenbach, M. (Eds.), Embedded Computer Systems: Architectures, Modeling, and Simulation. Springer Nature Switzerland, Cham, ISBN: 978-3-031-46077-7, pp. 494–501.

Queiroz, A.L., Mckeever, S., Keegan, B., 2019. Eavesdropping hackers: Detecting software vulnerability communication on social media using text mining. In: The Fourth International Conference on Cyber-Technologies and Cyber-Systems. pp. 41–48.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., Liu, P.J., 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. J. Mach. Learn. Res. 21 (140), 1–67.

Rahman, M.R., Basak, S.K., Mahdavi-Hezaveh, R., Williams, L.A., 2024. Attackers reveal their arsenal: An investigation of adversarial techniques in CTI reports. CoRR.

Rahman, M.R., Hezaveh, R.M., Williams, L., 2023. What are the attackers doing now? Automating cyberthreat intelligence extraction from text on pace with the changing threat landscape: A survey. ACM Comput. Surv. 55 (12), 1–36.

Ramos, J., et al., 2003. Using tf-idf to determine word relevance in document queries. In: Proceedings of the First Instructional Conference on Machine Learning, vol. 242, (1), Citeseer, pp. 29–48.

Rapid7, (2025). Metasploit. https://www.metasploit.com/.

Refat, O., Bruno, R., Barbara, R., 2024. A comparison of vulnerability feature extraction methods from textual attack patterns. In: 2024 50th Euromicro Conference on Software Engineering and Advanced Applications. SEAA, IEEE.

Reimers, N., Gurevych, I., 2019. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). Association for Computational Linguistics.

Robert Muggah, M.M., 2023. Cybercrime to cost the world 10.5 trillion annually by 2025. https://www.weforum.org/agenda/2023/01/global-rules-crack-down-cybercrime/.

Satvat, K., Gjomemo, R., Venkatakrishnan, V., 2021. Extractor: Extracting attack behavior from threat reports. In: 2021 IEEE European Symposium on Security and Privacy (EuroS&P). IEEE, pp. 598–615.

SecurityWeek, (2025). News Vulnerabilities. https://www.securityweek.com/category/vulnerabilities/.

Shadish, W.R., Cook, T.D., Campbell, D.T., 2002. Experimental and quasi-experimental designs for generalized causal inference / william r. Shedish, Thomas d. Cook, donald t. Campbell. In: Experimental and Quasi-Experimental Designs for Generalized Causal Inference. Wadsworth, Cengage Learning, Belmont, CA, ISBN: 0395615569.

Sheng, V.S., Ling, C.X., 2006. Thresholding for making classifiers cost-sensitive. vol. 6, pp. 476–481.

Siino, M., Tinnirello, I., La Cascia, M., 2024. Is text preprocessing still worth the time? A comparative survey on the influence of popular preprocessing methods on transformers and traditional classifiers. Inf. Syst. 121, 102342.

Sjøberg, D.I., Bergersen, G.R., 2022. Construct validity in software engineering. IEEE Trans. Softw. Eng. 49 (3), 1374–1396.
Son, S.B., Park, S., Lee, H., Kim, Y., Kim, D., Kim, J., 2023. Introduction to MITRE att&ck: Concepts and use cases. In: 2023 International Conference on Information Networking. ICOIN, IEEE, pp. 158–161.
Sönmez, F.Ö., 2021. Classifying common vulnerabilities and exposures database using text mining and graph theoretical analysis. Mach. Intell. Big Data Anal. Cybersecurity Appl. 313–338.
Sun, J., Xing, Z., Guo, H., Ye, D., Li, X., Xu, X., Zhu, L., 2021. Generating informative CVE description from exploitdb posts by extractive summarization. CoRR.
Sun, X., Ye, Z., Bo, L., Wu, X., Wei, Y., Zhang, T., Li, B., 2023. Automatic software vulnerability assessment by extracting vulnerability elements. J. Syst. Softw. 204, 111790.
Tang, W., Tang, M., Ban, M., Zhao, Z., Feng, M., 2023. CSGVD: A deep learning approach combining sequence and graph embedding for source code vulnerability detection. J. Syst. Softw. 199, 111623.
Theisen, C., Munaiah, N., Al-Zyoud, M., Carver, J.C., Meneely, A., Williams, L., 2018. Attack surface definitions: A systematic literature review. Inf. Softw. Technol. 104, 94–103.
UKPLab, 2024. Sentence transformers. https://www.sbert.net/docs/pretrained_models.html.
UKPLab, (2025). Semantic Textual Similarity. https://github.com/UKPLab/sentence-transformers/blob/master/examples/sentence_transformer/training/sts/README.md.
Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., Zhou, M., 2020. Minilm: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. Adv. Neural Inf. Process. Syst. 33, 5776–5788.
Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B., Wesslén, A., et al., 2012. Experimentation in software engineering, vol. 236, Springer.
Wu, Y., Liu, Q., Liao, X., Ji, S., Wang, P., Wang, X., Wu, C., Li, Z., 2021. Price tag: towards semi-automatically discovery tactics, techniques and procedures of E-commerce cyber threat intelligence. IEEE Trans. Dependable Secur. Comput..

**Refat Othman** is a Ph.D. candidate in Advanced-Systems Engineering at the Free University of Bozen-Bolzano, specializing in cybersecurity. He works as a researcher at the Cybersecurity Laboratory (CSLab), where his research focuses on automating vulnerability detection from textual cyberattack descriptions using sentence transformer models, while leveraging datasets from MITRE repositories. Refat holds both a master's and bachelor's degree in Software Engineering and Computer Science from Birzeit University, where he also served as an instructor. His industry experience includes roles as a QA Team Leader and Senior Software Engineer at NVIDIA and Asal Technologies.

**Diaeddin Rimawi** received his Ph.D. in Advanced-Systems Engineering from the Free University of Bozen-Bolzano, where he introduced the concept of green resilience in AI-enabled cyber–physical systems. Before his Ph.D., he worked in both academia and industry as a university instructor, head of instructors at a coding bootcamp, and software engineer. He is currently a Cybersecurity Technologist at the CSLab of the Free University of Bozen-Bolzano. His research interests include optimization, game theory, and reinforcement learning to support sustainable and resilient decision-making under unforeseen disruptions and adversarial attacks.

**Bruno Rossi** is Associate Professor at the Lab of Software Architectures and Information Systems, Faculty of Informatics, Masaryk University, Brno, Czech Republic. In 2008 he received the Ph.D. degree in Computer Science from the Free University of Bozen-Bolzano, Bolzano, Italy. He is part of the C4e project, "Center of Excellence for Cybercrime, Cybersecurity and Protection of Critical Information Infrastructures," and has taken part to key European (COSPA, STREP FP6) and Italian projects (ArtDeco, FIRB 36 months). He is involved as an Active Member in several journal and conference program committees in software engineering. His research interests include empirical software engineering research and cyberphysical systems, with specific focus on smart grids.

**Barbara Russo** is a Full Professor of Computer Science at the Faculty of Engineering at the Free University of Bozen-Bolzano. She has served as Vice Dean for Research at the Faculty of Computer Science and Technologies and has coordinated scientific programs for international symposia and doctoral schools. Barbara Russo is an Associate Editor for the International Journal of Information and Software Technology (Elsevier) and acts as a reviewer for several high-level journals and conferences in software engineering e.g., ICSE, MSR, EMSE, FSE) journals (TSE, TOSEM, ESE, ASE) in Software Engineering. She founded and has been organizing the international doctoral school in software engineering in collaboration with the University of Innsbruck for the past 10 years, with the 2023 edition focusing on cybersecurity. She has published 150 articles in international mathematics and computer science journals, which have received over 3500 citations (h index=29). Her expertise lies in software development and maintenance aimed at ensuring systems with high standards of reliability, scalability, performance, and security. In the last four years, prof. Russo' research specializes in data extraction using Deep Learning techniques to reconstruct system and user behavior and detect anomalies and code vulnerabilities.

Since 2023 she is coordinating the Cyber Security Lab of the Free University of Bolzano, supported by the FESR 2021–2027 program. Since 2024 she is coordinating the Ph.D. program in Advanced Systems Engineering of the Faculty of Engineering of the Free University of Bozen-Bolzano, Italy. In 2020–2023, she was the vice-dean for research at the Faculty of Computer Science of the Free University of Bozen-Bolzano, Italy Since 2022 she is unit coordinator of BeT project funded by PRIN ministerial fund and she was unit coordinator of other two PRIN projects (Idea and GAUSS). In 2014–2019 she was project coordinator of the Erasmus Mundus in Software Engineering of the Faculty of computer Science of the Free University of Bozen-Bolzano, Italy (successfully audited by the European Commission in 2022) (2M EURO).